Northeastern University

Project Report – *Image Captioning with CLIP and GPT-2*

(Generative Project Milestone - 1)

IE 7615 - Deep Learning for AI

Spring 2026

03/22/2026

Group 3

Rithika Sankar Rajeswari 002311800

Meena Periasamy 002311852

Sreevarshan Sathiyamurthy 002311854

Sahil Mohanty 002317489

1. Dataset

Dataset: COCO Captions — Karpathy Split

Source: yerevann/coco-karpathy (HuggingFace Hub)

We chose COCO Captions because it is the most widely used benchmark in image captioning research. The Karpathy split provides a standardized train/validation/test partition used across the literature, making our results directly comparable to published work. The dataset is hosted in parquet format on HuggingFace, allowing seamless loading without custom scripts.

Full dataset: 82,783 training images, 5,000 validation, 5,000 test — each with 5 human-written captions.

Milestone 1 subset: 2,500 image-caption pairs randomly sampled from the training split.

Preprocessing pipeline:

1. Random sampling with fixed seed (42) for reproducibility; 2× oversampling to handle download failures
2. Images downloaded from COCO URLs, converted to RGB, and filtered by minimum size (64×64 pixels)
3. Captions lowercased and stripped of special characters (retaining alphanumeric, basic punctuation, hyphens, apostrophes)
4. Whitespace normalized to single spaces
5. Captions outside the 4–60 word range filtered out
6. Final result: 2,500 valid image-caption pairs with 0 skipped

---

2. Model

Our system has three components:

Image Encoder — CLIP ViT-B/32 (Frozen) CLIP by OpenAI is a vision-language model pre-trained on 400M image-text pairs. We use the ViT-B/32 variant, which encodes each image into a 512-dimensional embedding vector through a Vision Transformer. CLIP remains frozen during training — its visual representations are already rich enough for our task.

Mapping Network (Trained) A 3-layer MLP that projects CLIP's 512-dim embedding into a sequence of 10 prefix tokens in GPT-2's 768-dimensional embedding space. This small network is the bridge between the vision and language components, and is trained from scratch.

Architecture: Linear(512→512) → GELU → LayerNorm → Linear(512→512) → GELU → LayerNorm → Linear(512→7680), reshaped to 10×768.

Caption Generator — GPT-2 with LoRA (Fine-tuned) GPT-2 (124M parameters) generates captions autoregressively, conditioned on the prefix tokens from the mapping network. We apply LoRA (Low-Rank Adaptation) to the attention layers, reducing trainable parameters to ~0.5–1% of the model. This makes training feasible on Google Colab GPUs.

Pipeline:

Image → CLIP ViT-B/32 → [512-dim embedding]
↓
Mapping Network (MLP)
↓
[10 × 768-dim prefix tokens]
↓
GPT-2 (LoRA) generates caption tokens
↓
"a dog playing in the park"

3. Environment Setup

Tool	Version / Detail
Python	3.12
PyTorch	2.1+ (MPS backend for local dev, CUDA for Colab)
HuggingFace Transformers	4.36+
HuggingFace Datasets	2.16+
PEFT	LoRA parameter-efficient fine-tuning
Local hardware	MacBook Air M3 (Apple MPS)
Training hardware	Google Colab GPU (T4)

All dependencies are listed in `requirements.txt`. The environment was tested on macOS with Apple Silicon using the MPS backend for GPU acceleration.

---

4. Baseline Pipeline — Image → Embedding + Caption Tokenization

The Milestone 1 pipeline demonstrates two core capabilities:

Image → CLIP Embedding: Each image is processed through CLIP's vision encoder and visual projection layer, producing a 512-dimensional embedding vector. These embeddings capture the semantic content of each image and will serve as input to the mapping network during training.

Caption → GPT-2 Tokens: Each caption is tokenized using GPT-2's BPE tokenizer with a maximum sequence length of 64 tokens. Padding uses the EOS token (GPT-2 has no native pad token). A roundtrip check (tokenize → decode) verifies that the original caption is recoverable.

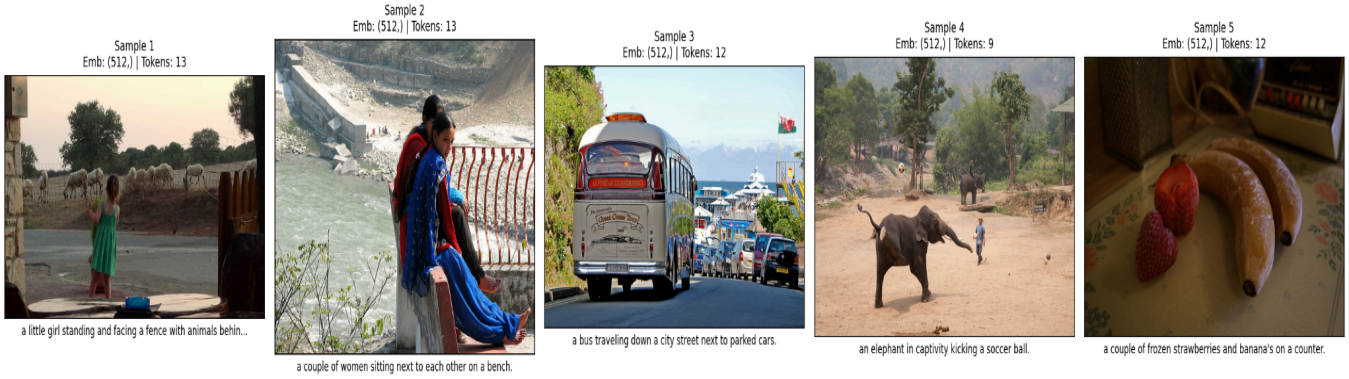
---

5. Sample Test Runs (5 Image-Caption Pairs)

Five randomly selected pairs were processed through the full pipeline. All passed the roundtrip verification check.

Sample	COCO ID	Caption	Embedding	Tokens
1	243829	a little girl standing and facing a fence with animals behind it.	512-dim	13
2	151124	a couple of women sitting next to each other on a bench.	512-dim	13
3	222970	a bus traveling down a city street next to parked cars.	512-dim	12
4	79957	an elephant in captivity kicking a soccer ball.	512-dim	9
5	538776	a couple of frozen strawberries and banana's on a counter.	512-dim	12

Output:



Team & Roles

Member	Role	Responsibilities
Meena Periasamy	Data Pipeline & Embeddings	Dataset selection and preprocessing, CLIP embedding pipeline, environment setup
Rithika Sankar Rajeswari	Model Architecture	GPT-2 tokenizer integration, model research and architecture planning
Sreevarshan Sathiyamurthy	Training & Evaluation	Environment testing, pipeline validation, sample test runs
Sahil Mohanty	Inference & Presentation	Project proposal, README documentation, GitHub repo setup

---

6. GitHub Repository

URL: github.com/heisenberg1804/image-captioning-with-CLIP-and-GPT-2

```
├── README.md                 # Project overview and setup instructions
├── requirements.txt          # Python dependencies
├── docs/
│   └── proposal.md           # This document
├── src/
│   ├── config.py             # Central configuration (paths, models, device)
│   ├── data_preprocessing.py # Dataset download, cleaning, subset creation
│   ├── embedding_pipeline.py # CLIP image embedder + GPT-2 tokenizer
│   └── run_samples.py        # Milestone 1 test runs + visual grid
├── data/
│   └── hf_cache/             # HuggingFace download cache (gitignored)
└── outputs/
    └── sample_test_runs.png  # Visual grid of 5 sample test runs
```

---

7. Milestone Summary

Deliverable	Status
Dataset chosen (COCO Captions) and preprocessed (2,500 pairs)	✅
Environment set up (PyTorch + HuggingFace Transformers + CLIP)	✅
Baseline script: image → embedding pipeline	✅
Caption tokenization using GPT-2 tokenizer	✅
5 sample image:caption test runs	✅
Project proposal (front page + dataset + model + roles)	✅
GitHub repo initialized	✅